



ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

Python Framework with FastAPI

Eng. Fatma Elraey

Lab 4 — Migrate, Modularize & Document the SQLite API

Field	Detail
Instructor	Eng. Fatma Elraey
Course	Python Framework with FastAPI
Session	Lab 4 of 5
Focus	Practice one Alembic schema change on the same documents.db file, move CRUD routes to APIRouter while keeping Session(engine), and improve FastAPI's generated API docs.
Evaluation	Lab Assignment — 100%

Before You Start

This lab starts from the simplified Lecture 3 project: the same documents.db SQLite file, the same Document ORM model, and the same direct with Session(engine) as db CRUD pattern. Do not replace that database pattern in this lab. We only add one migration, move routes into APIRouter, and improve the generated docs.

Today's Lab Plan

- 1. Prepare the Lecture 3 project and install Alembic
- 2. Connect Alembic to Base.metadata
- 3. Add an optional description column and migrate documents.db
- 4. Move CRUD endpoints into APIRouter and keep Session(engine)
- 5. Improve Swagger/ReDoc metadata and documented responses
- 6. Verify old data, /docs and /redoc, then submit

Install

```
pip install alembic
```

01

Connect Alembic to the Lecture 3 Project

Goal: Keep the Lecture 3 SQLite database, make the ORM metadata importable, and connect Alembic to it.

Steps

1. Keep the existing documents.db file from Lab 3.
2. Move engine + Base to database.py and keep Document in models.py.
3. Run alembic init alembic and point alembic.ini to sqlite:///documents.db.
4. Import Base + models in alembic/env.py and set target_metadata = Base.metadata.

Starter / reference code

```
# command line
alembic init alembic

# alembic.ini
sqlalchemy.url = sqlite:///documents.db

# alembic/env.py
from database import Base
import models

target_metadata = Base.metadata
```

✓ CHECKPOINT — Exercise 1

Q1. Why import models before autogenerate?

Answer: The Document mapping must be registered in Base.metadata so Alembic can compare it with documents.db.

02

Generate & Apply One Migration

Goal: Make one safe schema change to the existing Lecture 3 database without losing its rows.

Steps

1. Add description as an optional String(250) column on Document.
2. Run alembic revision --autogenerate -m "add description".
3. Review the generated upgrade() before applying it.
4. Run alembic upgrade head, then alembic current.
5. GET an old document and confirm the existing data is still available.

Starter / reference code

```
# models.py – add one optional field
description: Mapped[str | None] = mapped_column(
    String(250),
    nullable=True,
)

# create + apply
alembic revision --autogenerate -m "add description"
# review the generated migration file
alembic upgrade head
alembic current
```

✓ CHECKPOINT — Exercise 2

Q1. Which command applies the migration to documents.db?

Answer: alembic upgrade head.

03

Move CRUD Routes into APIRouter

Goal: Move route code out of main.py without changing the simple database Session pattern from Lecture 3.

Steps

1. Create routes/documents.py.
2. Create router = APIRouter(prefix="/documents", tags=["documents"]).
3. Move GET/POST/PUT/DELETE endpoints to the router and change @app to @router.
4. Keep with Session(engine) as db inside each database endpoint, then include the router in main.py.

Starter / reference code

```
# routes/documents.py
from fastapi import APIRouter
from sqlalchemy.orm import Session
from database import engine
from models import Document

router = APIRouter(prefix="/documents", tags=["documents"])

@router.get("")
def list_documents():
    with Session(engine) as db:
        return db.query(Document).all()

# main.py
from fastapi import FastAPI
from routes.documents import router as documents_router

app = FastAPI(title="Documents API", version="1.0.0")
app.include_router(documents_router)
```

✓ CHECKPOINT — Exercise 3

Q1. What database pattern should remain unchanged in this lab?

Answer: Use with Session(engine) as db inside the endpoints that touch the database.

04

Improve Swagger, ReDoc & Responses

Goal: Make the generated FastAPI documentation explain the same API clearly.

Steps

1. Add app title, description and version in main.py.
2. Add summary metadata to at least two Documents routes.
3. Keep response_model on public JSON endpoints where your schemas already support it.
4. Document at least one 404 response in a route decorator.
5. Open /docs, /redoc and /openapi.json and verify the route group.

Starter / reference code

```
app = FastAPI(
    title="Documents API",
    description="SQLite-backed JSON API for documents.",
    version="1.0.0",
)

@router.get(
   ("/{document_id}"),
    response_model=DocumentResponse,
    summary="Get one document",
    responses={404: {"description": "Document not found"}},
)
def get_document(document_id: int):
    with Session(engine) as db:
        ...
```

✓ CHECKPOINT — Exercise 4

Q1. Where can you interactively test the generated API documentation?

Answer: Open `/docs` for Swagger UI; `/redoc` is the reference-style view.

05**Submission Checklist**

- The original `documents.db` from Lab 3 is still used and old rows remain available.
- Alembic is configured with `target_metadata = Base.metadata`.
- The optional description migration was generated, reviewed and applied.
- Document CRUD endpoints live under an `APIRouter`.
- Database endpoints still use with `Session(engine)` as `db` and `main.py` includes the router.
- `/docs`, `/redoc` and `/openapi.json` clearly show the Documents routes, schemas and documented 404 behavior.